

Part A - Conditions

1. Write a program that classifies a number as positive, negative or zero.
2. Ask for an age and classify it into at least four age groups using if/elif/else.
3. Create a login check using a stored username and password. Both must match.
4. Given a score from 0-100, print a grade using at least five ranges. Think carefully about condition order.
5. Create a shipping rule based on order total and whether the customer is a member. Use and/or.
6. Write five expressions using ==, !=, >, <, >= and <= and predict each boolean result before running.

Part B - Truthy, falsy and membership

1. Create examples with empty string, non-empty string, zero, non-zero integer, empty list and non-empty list. Test each directly in an if statement.
2. Ask for a language and check whether it exists in a predefined list of supported languages.
3. Create a list of blocked usernames and reject a supplied username if it appears in the list.
4. Use not to express at least two conditions in a readable way.

Part C - For loops

1. Loop over a list of names and print a numbered greeting for each.
2. Loop over numbers 1-50 and print only even numbers.
3. Calculate the sum of a list manually using a loop rather than sum().
4. Find the largest number in a list manually without max().
5. Count how many words in a list have more than five characters.
6. Given a list of scores, count passes and failures using a threshold of 70.
7. Loop over a dictionary using keys, values and .items() in three separate examples.

Part D - range, enumerate and nested loops

1. Use range to print 10 down to 1.
2. Generate the multiplication table for a number supplied by the user.
3. Use enumerate to print a playlist with track numbers starting at 1.
4. Use nested loops to print coordinate pairs for x=1..3 and y=1..4.
5. Create a simple 5x5 text grid using nested loops.

Part E - While loops

1. Create a countdown from 10 to 0.
2. Ask repeatedly for a password until the correct password is entered.
3. Create a menu that repeats until the user chooses 'quit'. The menu can simply print which option was selected.
4. Ask the user for numbers until they enter 0. Keep a running total.
5. Create a guessing loop with a fixed secret number. Tell the user whether each guess is too high or too low.

Part F - break and continue

1. Loop through numbers 1-100 and stop when you reach the first number divisible by both 7 and 9.
2. Loop through a list of strings and skip empty strings using continue.
3. Search a list for a target name. Print 'found' and break when it appears; otherwise explain how you know it was not found.
4. Process a list of numeric values where negative values should be skipped and processing stops completely when the value 999 appears.

Part G - Applied challenge: Console study tracker

1. Create a list of dictionaries representing at least ten study sessions with subject and minutes.
2. Loop through the sessions and calculate total minutes.
3. Calculate total minutes per subject using a dictionary that starts empty and is updated inside the loop.
4. Identify the longest study session without `max(..., key=...)`.
5. Print only sessions longer than 45 minutes.
6. Create a repeated menu that lets a user: view all sessions, view total time, filter by subject, or quit.
7. Use `break/continue` where they genuinely improve the flow.

Part H - Stretch challenges

1. Print FizzBuzz from 1 to 100: multiples of 3 -> Fizz, 5 -> Buzz, both -> FizzBuzz.
2. Given a sentence, count vowels without using `.count()` repeatedly.
3. Find all duplicate values in a list using loops and collections.
4. Build a simple text histogram: for each number in [3, 5, 2], print that many * characters.